



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Title: Implementation of Longest Common Subsequence String Matching Algorithm

Objective: To compute longest common subsequence for the given two strings.

CO to be achieved:

CO 2	Analyze and solve problems for divide and conquer strategy, greedy method, dynamic programming approach and backtracking and branch & bound policies.
CO 3	Analyze and solve problems for different string matching algorithms.

Books/ Journals/ Websites referred:

1. Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press
2. T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algortihmts",2nd Edition ,MIT press/McGraw Hill,2001
3. <http://www.math.utah.edu/~alfeld/queens/queens>.

Pre Lab/ Prior Concepts:

Data structures, Concepts of algorithm analysis

Historical Profile:

Given 2 sequences, $X = x_1, \dots, x_m$ and $Y = y_1, \dots, y_n$, find a subsequence common to both whose length is longest. A subsequence doesn't have to be consecutive, but it has to be in order.

New Concepts to be learned:

String matching algorithm, Dynamic programming approach for LCS, Applications of LCS.



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Recursive Formulation:

Define $c[i, j]$ = length of LCS of X_i and Y_j .

Final answer will be computed with $c[m, n]$.

```
c[i, j] = 0
if i=0 or j=0.
c[i, j] = c[i - 1, j - 1] + 1
if i, j > 0 and  $x_i = y_j$ 

c[i, j] = max(c[i - 1, j], c[i, j - 1])
if i, j > 0 and  $x_i \neq y_j$ 
```

Algorithm: Longest Common Subsequence

Compute length of optimal solution-

LCS-LENGTH (X, Y, m, n)

```
for i ← 1 to m
  do c[i, 0] ← 0
for j ← 0 to n
  do c[0, j] ← 0
for i ← 1 to m
  do for j ← 1 to n
    do if  $x_i = y_j$ 
      then c[i, j] ← c[i - 1, j - 1] + 1
        b[i, j] ← "≈"
    else if c[i - 1, j] ≥ c[i, j - 1]
      then c[i, j] ← c[i - 1, j]
        b[i, j] ← "↑"
    else c[i, j] ← c[i, j - 1]
      b[i, j] ← "←"

return c and b
```

Print the solution-

PRINT-LCS(b, X, i, j)

```
if i = 0 or j = 0
  then return
if b[i, j] = "≈"
  then PRINT-LCS(b, X, i - 1, j - 1)
    print  $x_i$ 
elseif b[i, j] = "↑"
  then PRINT-LCS(b, X, i - 1, j)
else PRINT-LCS(b, X, i, j - 1)
```

Initial call is PRINT-LCS(b, X, m, n).

$b[i, j]$ points to table entry whose subproblem we used in solving LCS of X_i and Y_j .



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)

Department of Computer Engineering

When $b[i, j] = \approx$, we have extended LCS by one character. So longest common subsequence = entries with $\approx$ in them.

Example: LCS computation

Analysis of LCS computation



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Output:

Algorithm:

CONCLUSION: